

电子科技大学计算机科学与工程学
院（网络空间安全学院）

实 验 指 导 书

（实验）课程名称：区块链原理与开发（新生项目课程）

目录

(实验) 课程名称： 区块链原理与开发（新生项目课程） 1

一、 实验目的 1

二、 实验内容 1

三、 实验环境 1

四、 预备知识 1

五、 实验步骤 2

 5.1 GANACHE 和 METAMASK 的配置 2

 5.1.1 配置 Ganache 2

 5.1.2 配置 METAMASK 4

 5.2 智能合约的编译与部署 8

 5.2.1 编译智能合约 8

 5.2.2 部署智能合约 9

 5.3 前端页面设计与区块链交互 11

 5.4 结果展示 12

附录 14

 附录 1: HTML 代码 14

 附录 2: JS 代码 15

一、实验目的

通过区块链技术，实现一个简易投票系统。

二、实验内容

本实验通过 Ganache 搭建私链，使用 METAMASK 进行交互，随后使用 Remix 编写智能合约，最后将合约部署到前端，实现了一个简单的投票系统。

三、实验环境

具体开发环境有两种：一是 Geth + js，二是 Ganache + js。虽然我们可以用 Geth 搭一条私链进行开发，但以太坊比特币需要通过每个区块挖矿产生，块与块之间有一定的时间间隔，而且用 PC 挖矿时间长，产热高，使我们的开发效率比较低。所以本实验选择通过 Ganache 在内存中模拟区块链来开发。

其他工具：

1. METAMASK：以太坊钱包。能帮助用户方便地管理自己的以太坊数字资产。
2. Remix：开源的 Solidity 智能合约开发环境，提供基本的编译、部署至本地或测试网络、执行合约等功能。

四、预备知识

1. 区块链基础知识；
2. Solidity 语言；
3. 前端开发基础知识：HTML/CSS/JavaScript

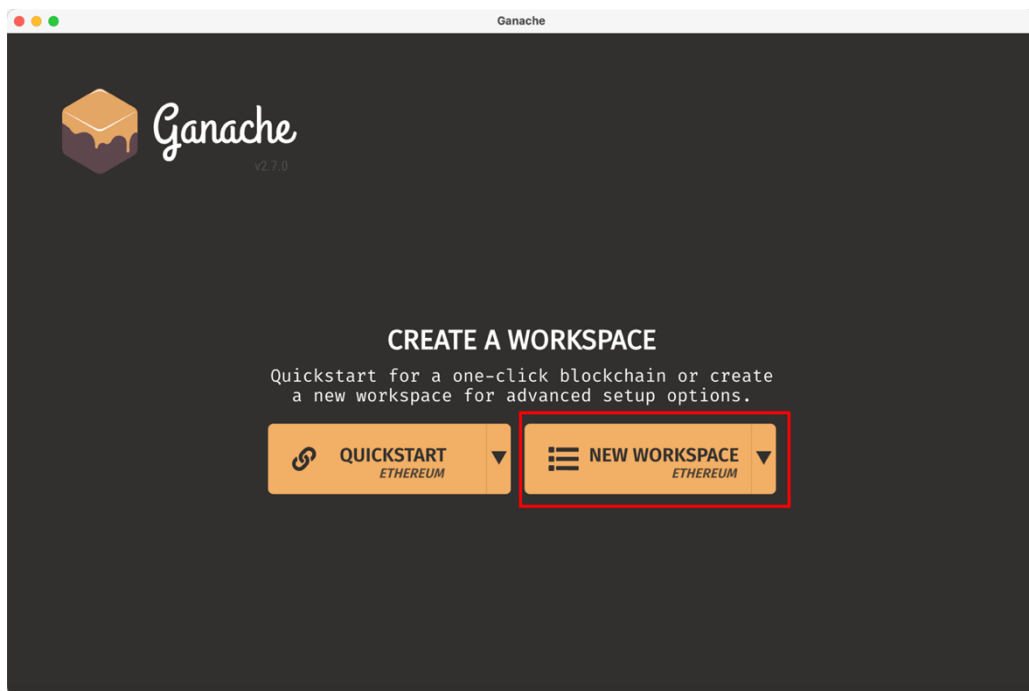
五、实验步骤

5.1 Ganache 和 METAMASK 的配置

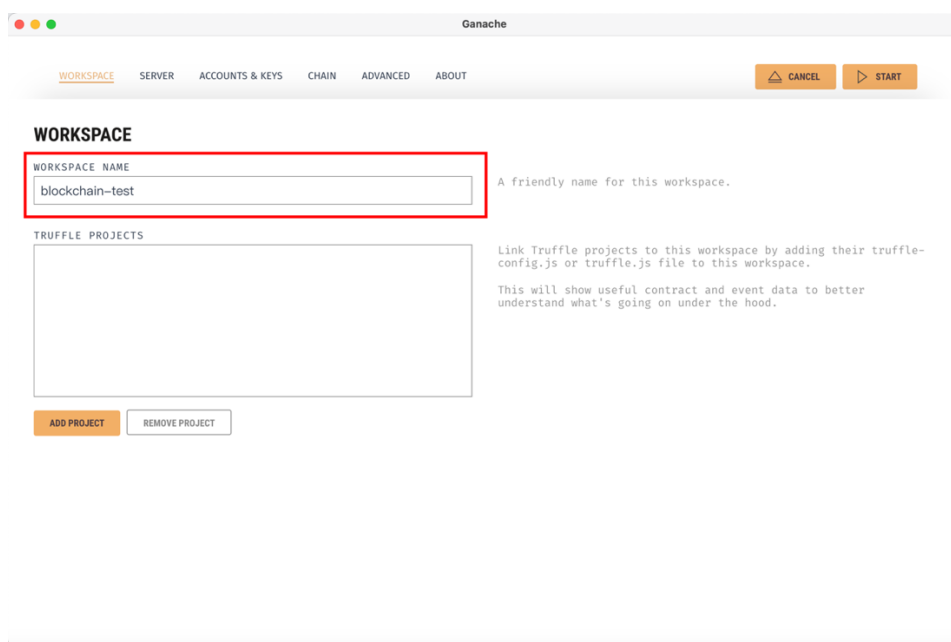
5.1.1 配置 Ganache

什么是 Ganache? Ganache 是一个用于本地开发的区块链，用于在以太坊区块链上开发去中心化的应用程序。Ganache 模拟了以太坊网络，你可以在发布到生产环境之前看到你的 DApp 将如何执行。

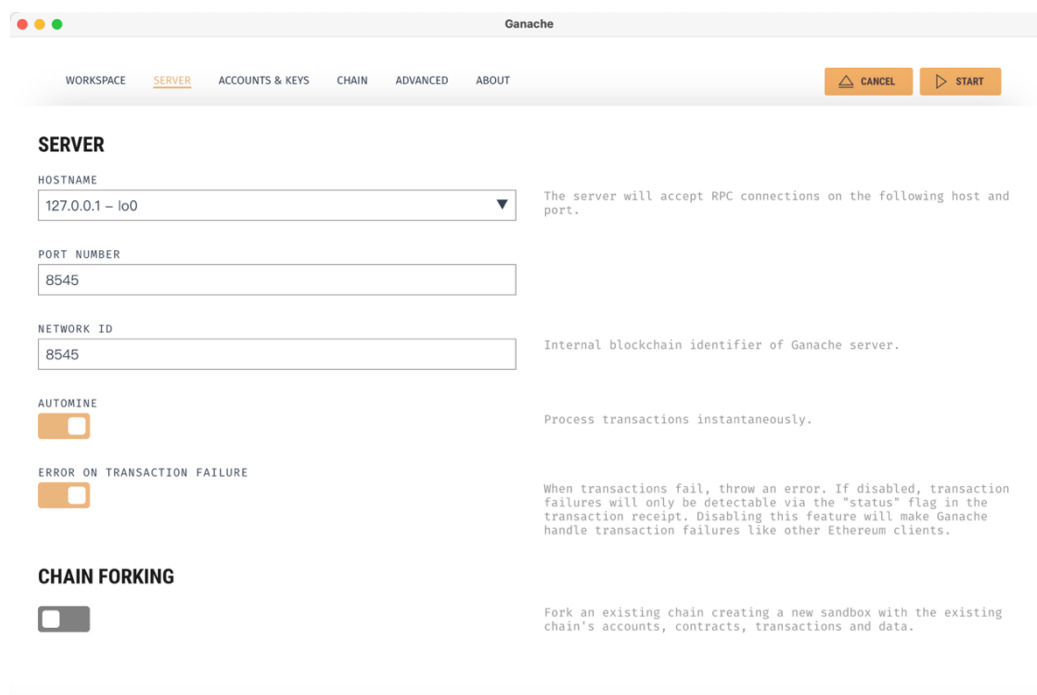
1. 下载 Ganache: <https://trufflesuite.com/ganache/>
2. 下载完成后，打开 Ganache，左边的按钮是快速启动，它的数据不会保存，每次启动后都是全新的开发环境。右边的按钮是保存当前的数据到相应的工作空间中，可以有多个工作空间。我们选择右边的 new workspace:



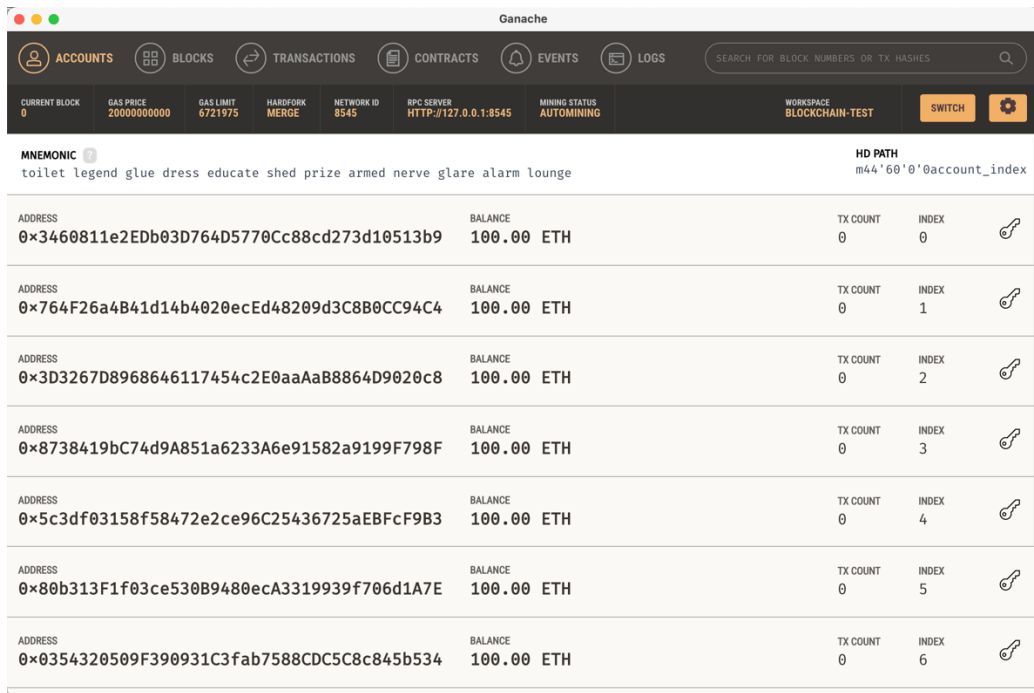
3. 为 workspace 取一个有意义的名字，这里我们设置为 blockchain-test:



4. 点击导航栏的 **server**，可以看到 RPC 服务相关设置。将 **PORT NUMBER** 修改为 8545，修改端口号的原因是为了后续能使私链与 **METAMASK** 正常连接，因为 **METAMASK** 中默认的 ID 为 8545：



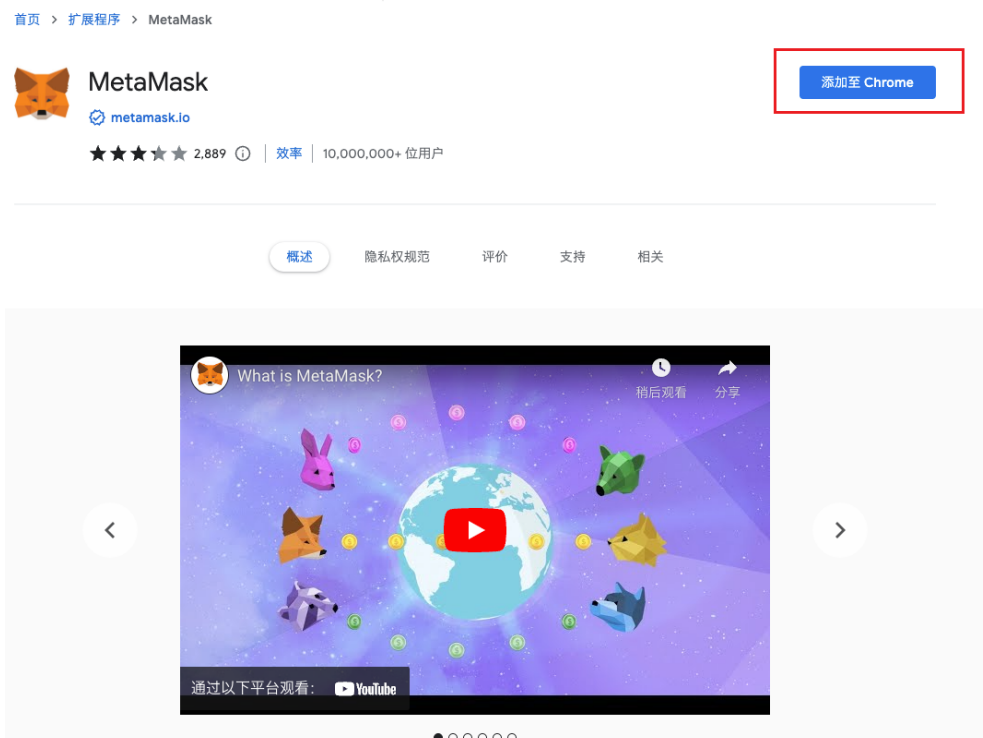
5. 点击导航栏的 **Accounts & Keys**，可以设置初始账号数量和账号的初始金额，我们保持默认。最后点击右上角的 **start**，我们的本地以太坊就可用了。搭建完成后，可以看到私链中自动生成了存有以太币的十个账户：



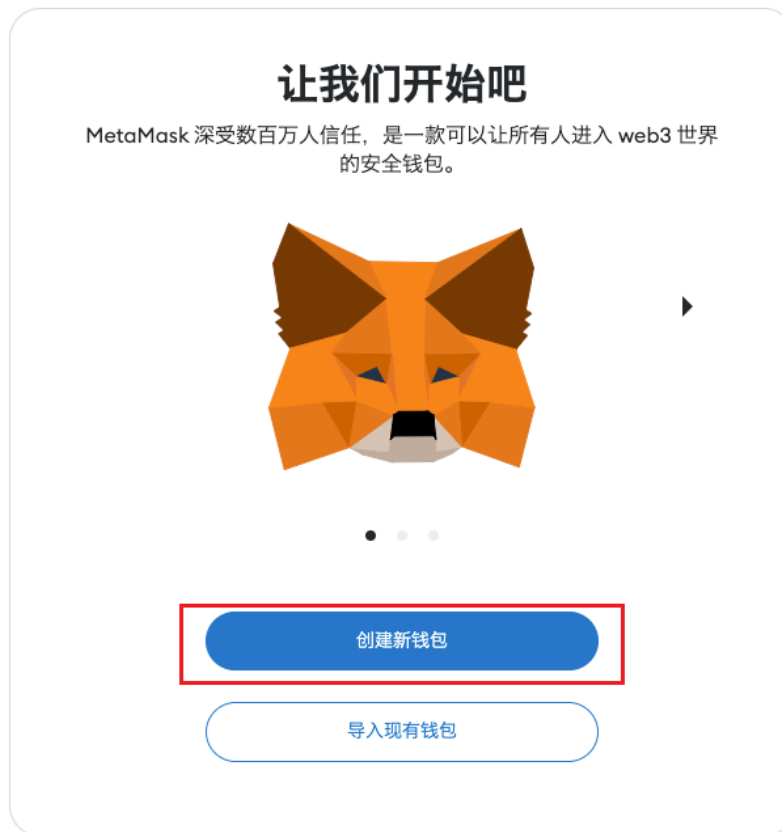
5.1.2 配置 METAMASK

什么是 METAMASK? METAMASK 是一个轻量化的以太坊钱包，支持私人网络及自定义 RPC。在部署和调用合约时，需要消耗 gas，我们使用 Ganache 提供的本地私链和虚拟账户通过 METAMASK 来模拟这一步骤。

1. Google Chrome 浏览器安装 METAMASK 插件：<https://metamask.io/>



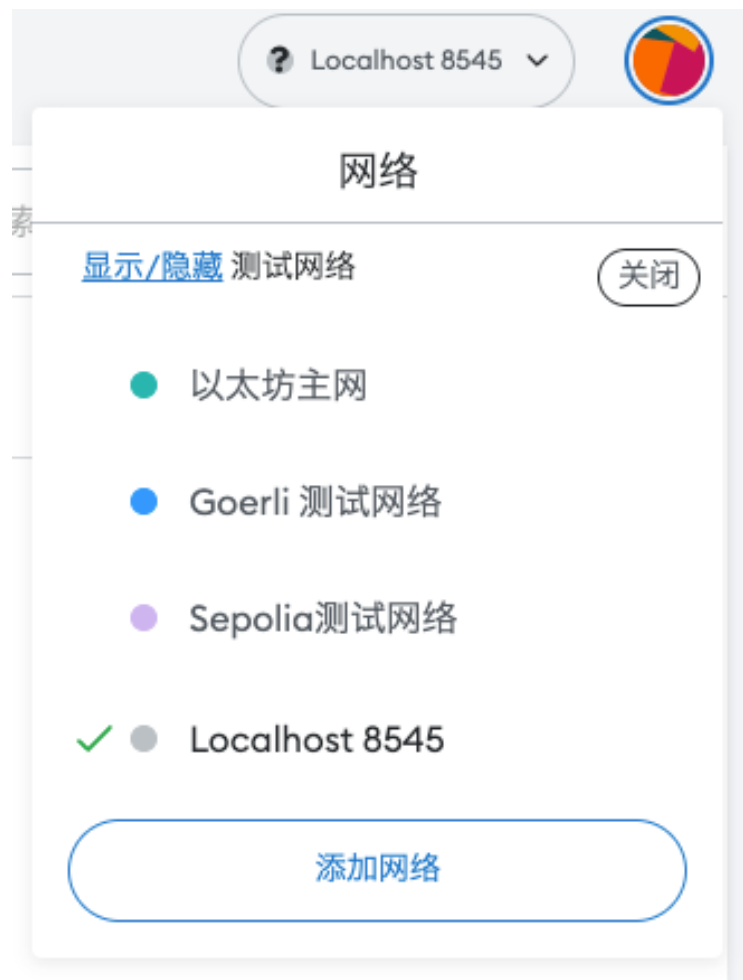
2. 打开 METAMASK 插件，选择创建新钱包



3. 创建完成之后，点击浏览器右上角的 METAMASK 图标，显示测试网络：



4. 显示测试网络之后，将网络设置为 Localhost 8545



5. 复制 Ganache 生成的 10 个账户中的某一个私钥，这里选择第一个：

Ganache

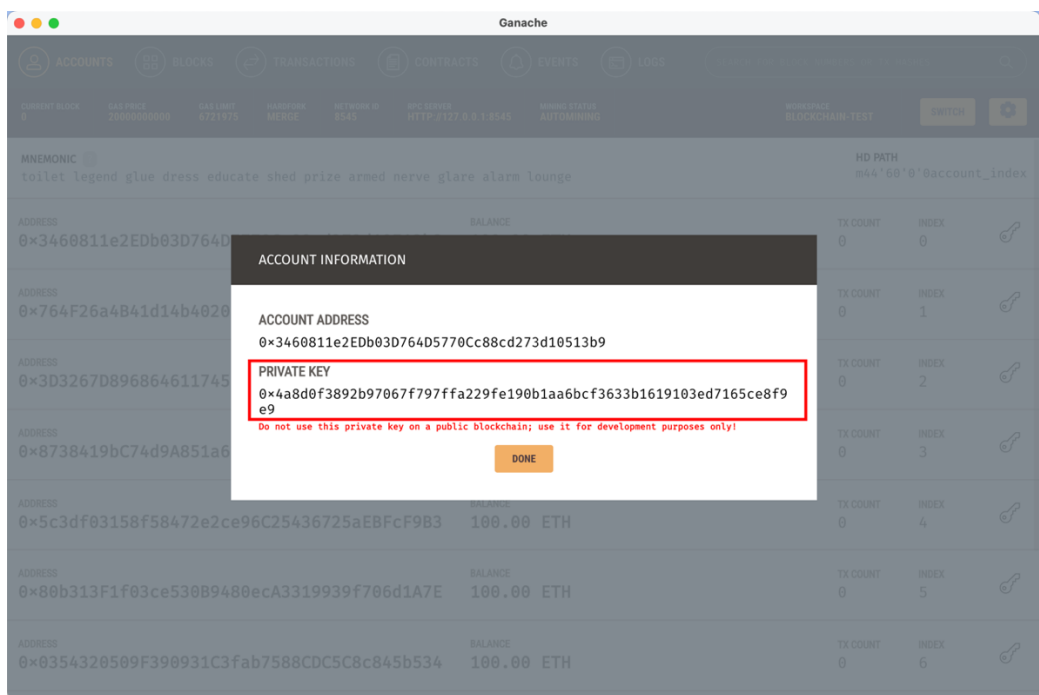
ACCOUNTS BLOCKS TRANSACTIONS CONTRACTS EVENTS LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

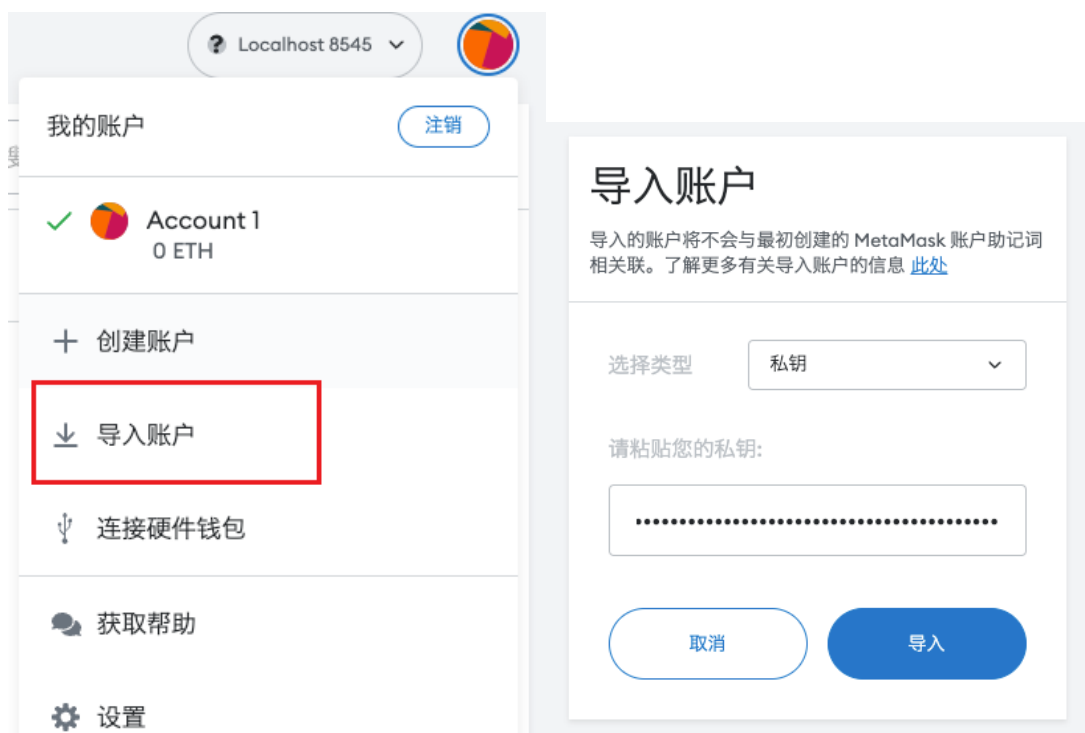
CURRENT BLOCK 0 GAS PRICE 20000000000 GAS LIMIT 6721975 HARDFORK MERGE NETWORK ID 8545 RPC SERVER HTTP://127.0.0.1:8545 MINING STATUS AUTOMINING WORKSPACE BLOCKCHAIN-TEST SWITCH

MNEMONIC ? toilet legend glue dress educate shed prize armed nerve glare alarm lounge HD PATH m44'60'0'0account_index

ADDRESS	BALANCE	TX COUNT	INDEX	
0x3460811e2EDb03D764D5770Cc88cd273d10513b9	100.00 ETH	0	0	
0x764F26a4B41d14b4020ecEd48209d3C8B0CC94C4	100.00 ETH	0	1	
0x3D3267D8968646117454c2E0aaAaB8864D9020c8	100.00 ETH	0	2	
0x8738419bC74d9A851a6233A6e91582a9199F798F	100.00 ETH	0	3	
0x5c3df03158f58472e2ce96C25436725aEBFcF9B3	100.00 ETH	0	4	
0x80b313F1f03ce530B9480ecA3319939f706d1A7E	100.00 ETH	0	5	
0x0354320509F390931C3fab7588CDC5C8c845b534	100.00 ETH	0	6	



6. 打开 METAMASK 插件，选择导入账户，粘贴刚刚复制的私钥：



7. 导入成功后可以看到下图：



5.2 智能合约的编译与部署

5.2.1 编译智能合约

什么是 Solidity? Solidity 是一门面向合约的、为实现智能合约而创建的高级编程语言。这门语言受到了 C++, Python 和 Javascript 语言的影响, 能在以太坊虚拟机 (EVM)上运行。

1. 在本实验中, 我们使用 solidity 实现智能合约的编写。以下是具体的代码:

```
pragma solidity 0.4.22;
contract Voting {
    bytes32[] public candidateList;

    mapping (bytes32 => uint8) public votesReceived;

    constructor(bytes32[] memory candidateNames) public {
        candidateList = candidateNames;
    }

    function voteForCandidate(bytes32 candidate) public {
        require(validCandidate(candidate));
        votesReceived[candidate] += 1;
    }

    function totalVotesFor(bytes32 candidate) view public returns (uint8) {
        require(validCandidate(candidate));
        return votesReceived[candidate];
    }
}
```

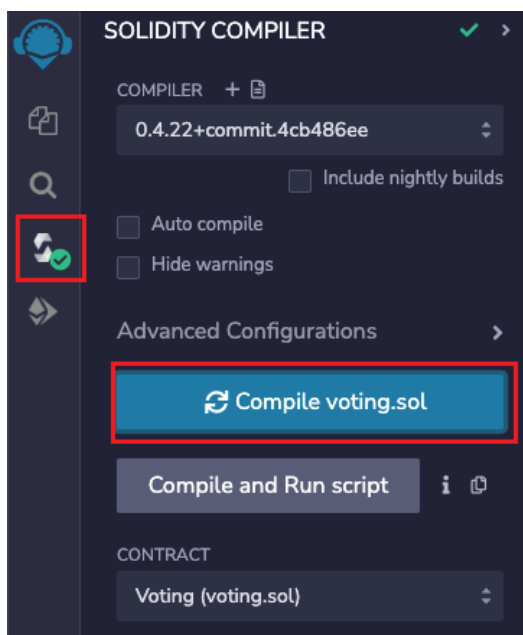
```

    }

    function validCandidate(bytes32 candidate) view public returns (bool) {
        for(uint8 i = 0; i < candidateList.length; i++) {
            if(candidateList[i] == candidate)
                return true;
        }
        return false;
    }
}

```

2. 使用 Remix 实现智能合约的编译: <http://remix.ethereum.org/> (Remix 是以太坊的一个在线集成开发环境)
3. 打开 Remix 之后, 在 contracts 文件夹里创建一个后缀为 voting.sol 的文件, 将合约代码复制进去, 然后编译, 编译成功如下图:



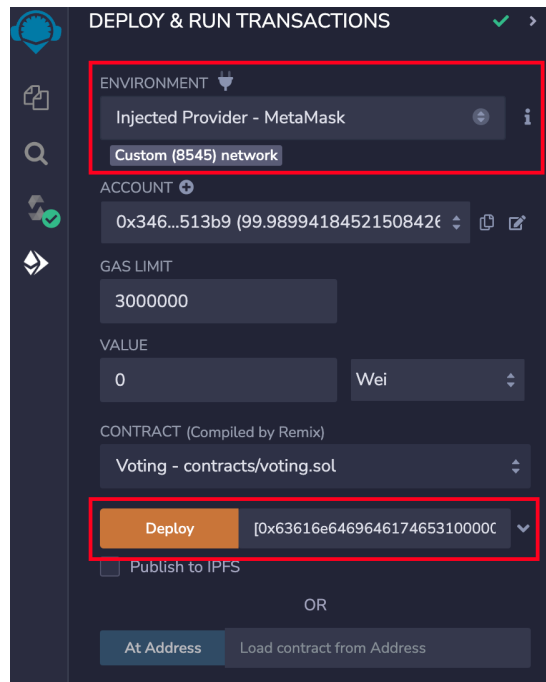
5.2.2 部署智能合约

1. 首先将 Remix 中的 ENVIRONMENT 选成 Injected Web3;
2. 接着在 Deploy 按钮后面输入相关参数(候选人名字的 byte32 形式)将智能合约部署。本实验后面的示例使用了三个候选人(candidate1,candidate2,candidate3), 转换为 Byte32 类型后形式如下, 直接填入 Deploy 按钮后面的输入框即可:

```

[0x63616e6469646174653100000000000000000000000000000000000000000000,
0x63616e6469646174653200000000000000000000000000000000000000000000,
0x63616e6469646174653300000000000000000000000000000000000000000000]

```

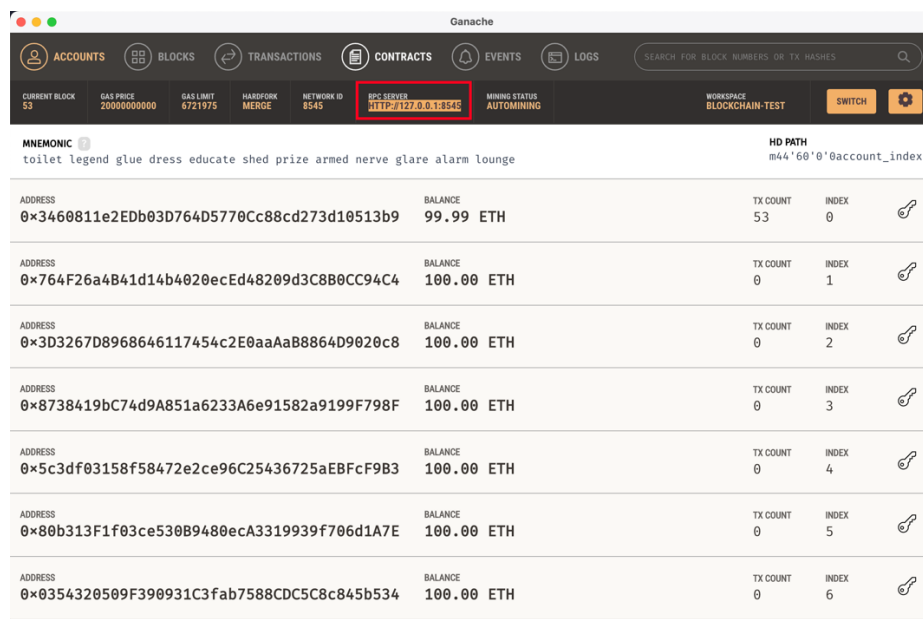


3. 点击 Deploy 按钮后，METAMASK 会自动跳出来，点击“确定”：

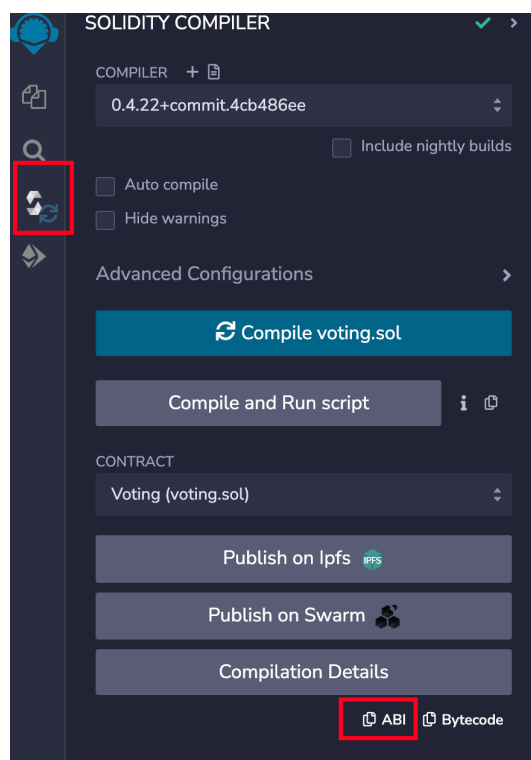


5.3 前端页面设计与区块链交互

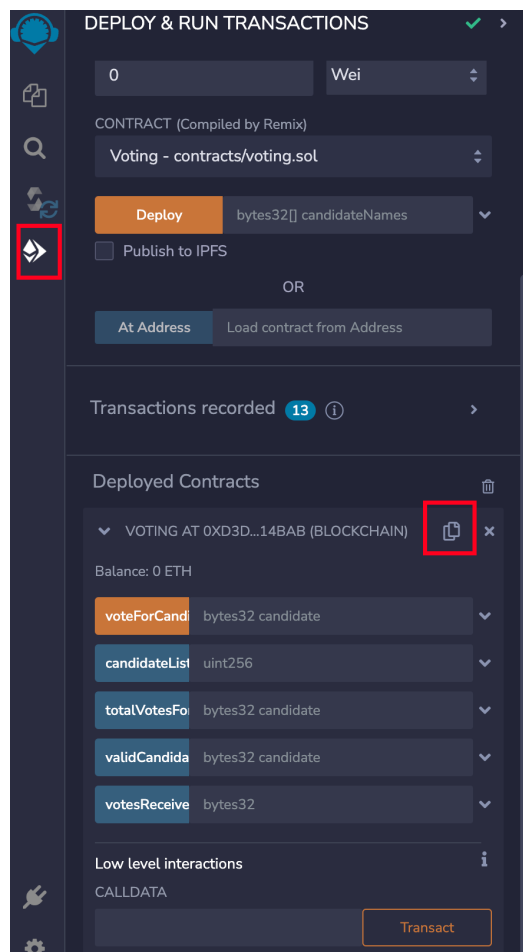
1. 本实验通过 HTML 实现前端页面设计，并与区块链实现交互。HTML 代码见附录 1；
2. HTML 中通过 Javascript 来实现调用区块链中的方法（上述 HTML 代码引用的是 index.js 文件，因此需要将区块链的交互逻辑添加到名为 index.js 的文件中，并与本 HTML 文件放置于同一目录下，js 代码中需要修改的三个变量为 server、abi、address，js 代码见附录 2；
3. 获取 server: Ganache 的 RPC SERVER 复制过来即可



4. 获取 abi: Remix 中复制



5. 获取 address: Remix 中复制



5.4 结果展示

1. 使用谷歌浏览器直接打开 `helloDapp.html` 文件, 可以看到初始化的时候每个候选人的投票数都为 0:

← → ↻ ⓘ 文件 | /Users/orange/code/blockchain/helloDapp.html#

简易投票应用

候选人	选票数
Alice	0
Bob	0
Cary	0

2. 为 Alice 投票, 点击投票后可以看到 Alice 的票数增 1:

简易投票应用

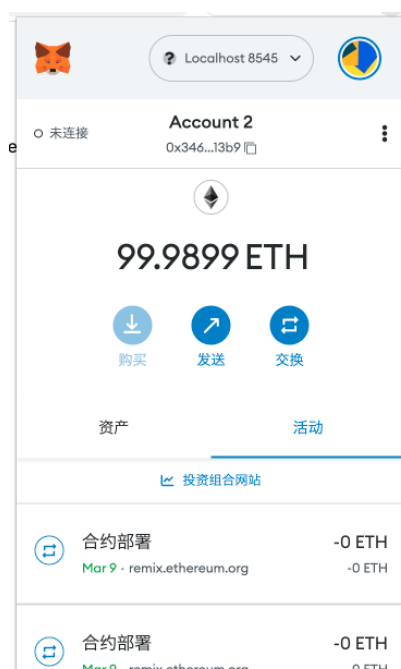
候选人	选票数
Alice	1
Bob	0
Cary	0

Alice

3. 此时去查看 Ganache，可以看到 TX COUNT 也增一：

Ganache			
ACCOUNTS	BLOCKS	TRANSACTIONS	CONTRACTS
CURRENT BLOCK 56	GAS PRICE 2000000000	GAS LIMIT 6721975	HARDFORK MERGE
NETWORK ID 8545	RPC SERVER HTTP://127.0.0.1:8545	MINING STATUS AUTOMINING	WORKSPACE BLOCKCHAIN-TEST
MNEMONIC toilet legend glue dress educate shed prize armed nerve glare alarm lounge		HD PATH m/44'/60'/0'/0/account_index	
ADDRESS 0x3460811e2EDb03D764D5770Cc88cd273d10513b9	BALANCE 99.99 ETH	TX COUNT 56	INDEX 0
ADDRESS 0x764F26a4B41d14b4020ecEd48209d3C8B0CC94C4	BALANCE 100.00 ETH	TX COUNT 0	INDEX 1
ADDRESS 0x3D3267D8968646117454c2E0aaAaB8864D9020c8	BALANCE 100.00 ETH	TX COUNT 0	INDEX 2
ADDRESS 0x8738419bC74d9A851a6233A6e91582a9199F798F	BALANCE 100.00 ETH	TX COUNT 0	INDEX 3
ADDRESS 0x5c3df03158f58472e2ce96C25436725aEBFcF9B3	BALANCE 100.00 ETH	TX COUNT 0	INDEX 4
ADDRESS 0x80b313F1f03ce530B9480ecA3319939f706d1A7E	BALANCE 100.00 ETH	TX COUNT 0	INDEX 5
ADDRESS 0x0354320509F390931C3fab7588CDC5C8c845b534	BALANCE 100.00 ETH	TX COUNT 0	INDEX 6

4. 打开 METAMASK 会观察到每一次投票会消耗 gas：



附录

附录 1: html 代码

helloDApp.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Hello World DApp</title>
  <link href='https://fonts.googleapis.com/css?family=Open+Sans:400,700' rel='stylesheet' type='text/css'>
  <link href='https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css' rel='stylesheet'
type='text/css'>
</head>
<body class="container">
  <h1>简易投票应用</h1>
  <div class="table-responsive">
    <table class="table table-bordered">
      <thead>
        <tr>
          <th>候选人</th>
          <th>选票数</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>Alice</td>
          <td id="candidate1"></td>
        </tr>
        <tr>
          <td>Bob</td>
          <td id="candidate2"></td>
        </tr>
        <tr>
          <td>Cary</td>
          <td id="candidate3"></td>
        </tr>
      </tbody>
    </table>
  </div>
  <input type="text" id="candidate" />
  <a href="#" onclick="voteForCandidate()" class="btn btn-primary">投票</a>
</body>
```

```
<script src="https://cdn.jsdelivr.net/npm/web3@0.20.1/dist/web3.js"></script>
<script src="https://code.jquery.com/jquery-3.1.1.slim.min.js"></script>
<script src="/index.js"></script>
</html>
```

附录 2: js 代码

```
// 连接以太坊节点, server 为 Ganache 的 RPC SERVER, 复制粘贴过来即可
server = "HTTP://127.0.0.1:8545"
web3 = new Web3(new Web3.providers.HttpProvider(server));
// 要使用智能合约, 必须先从区块链中获取到合约实例, 获取合约实例需要合约的 ABI 和合约的地址
abi = ;
address = ;
// 通过 ABI 和地址获取已部署的合约对象
VotingContract = web3.eth.contract(abi, address);
contractInstance = VotingContract.at(address);
candidates = {"Alice": "candidate1", "Bob": "candidate2", "Cary": "candidate3"}
candidates_strToBytes32 = {
  "candidate1": "0x63616e6469646174653100000000000000000000000000000000000000000000",
  "candidate2": "0x63616e6469646174653200000000000000000000000000000000000000000000",
  "candidate3": "0x63616e6469646174653300000000000000000000000000000000000000000000"
}
// 投票按钮的回调函数
function voteForCandidate() {
  candidateName = $("#candidate").val();
  contractInstance.voteForCandidate(candidates_strToBytes32[candidates[candidateName]], {from:
web3.eth.accounts[0]}, function() {
    let div_id = candidates[candidateName];
    $("##" + div_id).html(contractInstance.totalVotesFor.call(candidates_strToBytes32[div_id]).toString());
  });
}
// 页面初始化渲染
$(document).ready(function() {
  candidateNames = Object.keys(candidates);
  for (var i = 0; i < candidateNames.length; i++) {
    let name = candidateNames[i];
    let val = contractInstance.totalVotesFor.call(candidates_strToBytes32[candidates[name]]).toString()
    $("##" + candidates[name]).html(val);
  }
});
```
